

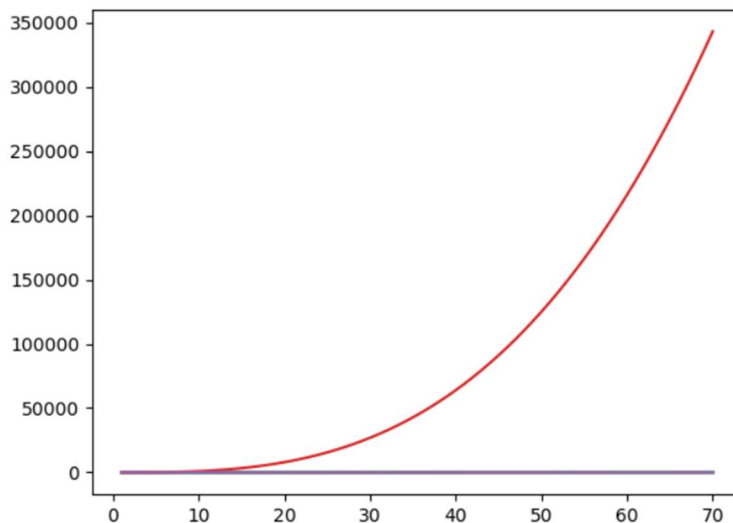
Lab 3: Data Visualization and Interpretation Using Matplotlib-II

Tasks:

1. Write code for line chart to generate 70 numbers with X- axis and Y-axis is $mx+c$, where $m=2$ and $c=0.3$, Draw the four different line charts using matplotlib library. (Take $Y1=X$, $Y2=X^2$, $Y3=X^3$ and $Y4=\sqrt{X}$).

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(1,71)
m = 2
c = 0.3
y = m * x + c
y1 = x
y2 = 2*x
y3= x**3
y4=np.sqrt(x)
plt.figure()
plt.plot(x,y,label = 'y= 2x+0.3')
plt.plot(x,y1,label = 'y= x')
plt.plot(x,y2,label='y=x^2')
plt.plot(x,y3,label = 'y= x^3')
plt.plot(x,y4,label='y=sqrt(x) ')
plt.show()
```

output:



2. Plot the function $y = x^2 - 4x + 3$ for x ranging from -2 to 6. Identify and annotate the minimum point of the graph. Add appropriate labels, title, and grid.

```

x = np.linspace(-2, 6, 400)
y = x**2 - 4*x + 3
xmin = 2
ymin = xmin**2 - 4*xmin + 3

plt.figure(figsize=(8, 5))
plt.plot(x, y, label=r"$y = x^2 - 4x + 3$", linewidth=2)

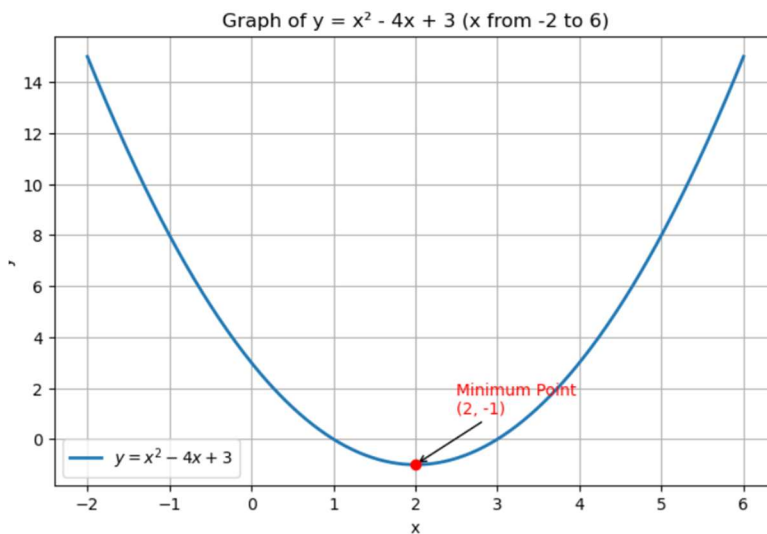
# Mark minimum point
plt.scatter(xmin, ymin, color="red", zorder=5)
plt.annotate(f"Minimum Point\n({xmin}, {ymin})",
            xy=(xmin, ymin),
            xytext=(xmin+0.5, ymin+2),
            arrowprops=dict(arrowstyle="->", color="black"),
            fontsize=10,
            color="red")

# Labels and title
plt.xlabel("x")
plt.ylabel("y")
plt.title("Graph of  $y = x^2 - 4x + 3$  (x from -2 to 6)")
plt.grid(True)
plt.legend()

# Show plot
plt.show()

```

output:



3. Plot $\sin(x)$ and $\cos(x)$ from 0 to 2π on the same graph. Use `fill_between()` to shade the area between the two curves where $\sin(x)$ is greater than $\cos(x)$. Add labels and a title

```
# Define x values from 0 to 2π
x = np.linspace(0, 2*np.pi, 500)

# Compute sin(x) and cos(x)
sin_y = np.sin(x)
cos_y = np.cos(x)

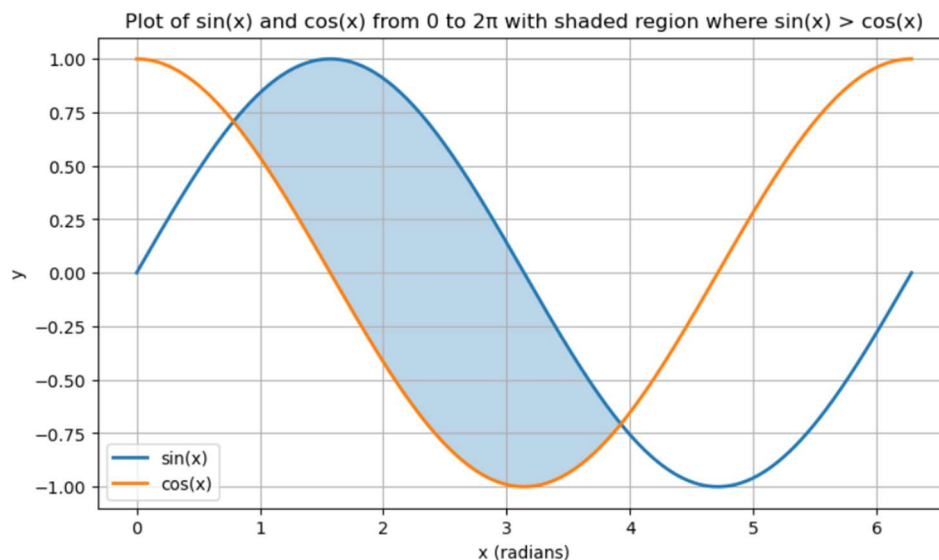
# Plot both curves
plt.figure(figsize=(9, 5))
plt.plot(x, sin_y, label="sin(x)", linewidth=2)
plt.plot(x, cos_y, label="cos(x)", linewidth=2)

# Shade area where sin(x) > cos(x)
plt.fill_between(x, sin_y, cos_y, where=(sin_y > cos_y), alpha=0.3,
interpolate=True)

# Labels and title
plt.xlabel("x (radians)")
plt.ylabel("y")
plt.title("Plot of sin(x) and cos(x) from 0 to 2π with shaded region
where sin(x) > cos(x)")
plt.legend()
plt.grid(True)

# Show plot
plt.show()
```

output:



4. Consider monthly temperature data for the year 2023 and plot it using a time-series line graph.

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

```
temperature = [12, 14, 18, 24, 30, 34, 32, 31, 28, 22, 16, 13]
```

Label the x-axis as Months and y-axis as Temperature (°C). Format the x-axis to show month names instead of numbers

```
# Data
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']
temperature = [12, 14, 18, 24, 30, 34, 32, 31, 28, 22, 16, 13]

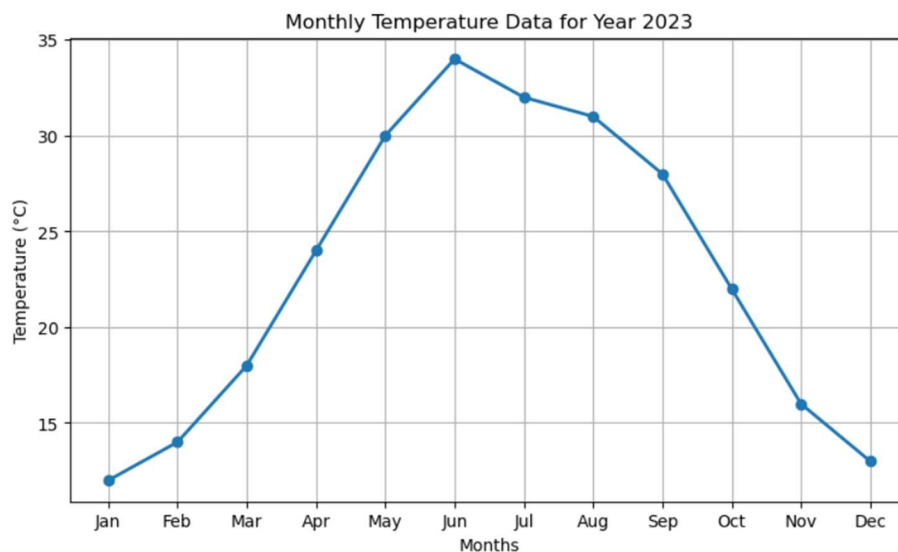
# Plot
plt.figure(figsize=(9, 5))
plt.plot(months, temperature, marker='o', linewidth=2)

# Labels and title
plt.xlabel("Months")
plt.ylabel("Temperature (°C)")
plt.title("Monthly Temperature Data for Year 2023")

# Grid
plt.grid(True)

# Show plot
plt.show()
```

output:



5. Plot three mathematical functions on the same graph:

$y=x^2$ (Red,DashedLine)

$y=2x+1$ (Blue,Solid Line)

$y=\sqrt{x}$ (Green, Dotted Line)

Add a legend, labels, and title

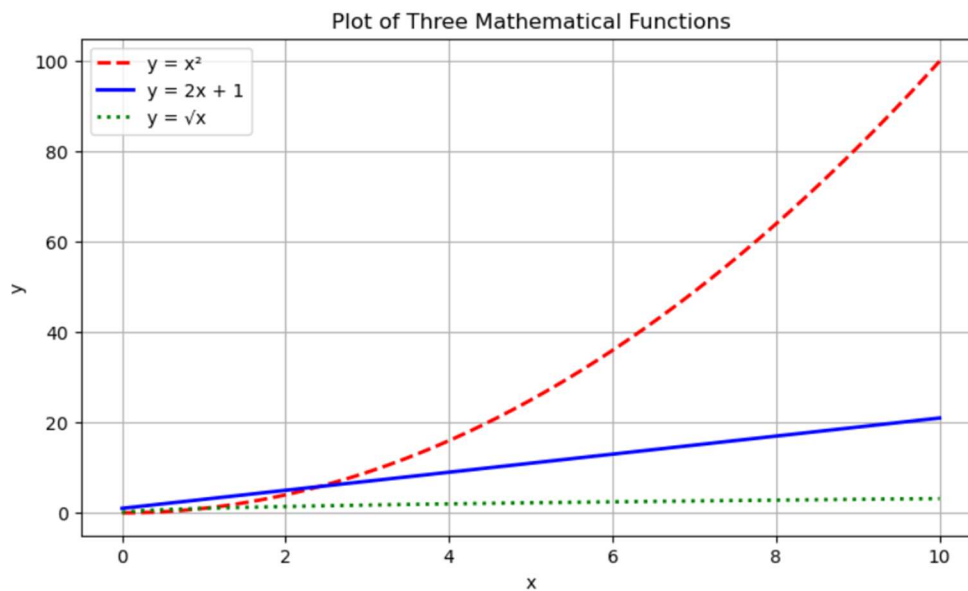
```
x = np.linspace(0, 10, 400)
y1 = x**2
y2 = 2*x + 1
y3 = np.sqrt(x)
plt.figure(figsize=(9, 5))

plt.plot(x, y1, 'r--', linewidth=2, label="y = x2")           # Red dashed
line
plt.plot(x, y2, 'b-', linewidth=2, label="y = 2x + 1")         # Blue solid
line
plt.plot(x, y3, 'g:', linewidth=2, label="y = √x")             # Green
dotted line

plt.xlabel("x")
plt.ylabel("y")
plt.title("Plot of Three Mathematical Functions")

plt.legend()
plt.grid(True)
plt.show()
```

output:



6. Create a dual-axis plot where:

The primary y-axis represents the monthly rainfall (in mm) in 2023 as a bar chart.

The secondary y-axis represents the average temperature (in °C) as a line graph on the same plot.

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

rainfall = [78, 62, 55, 41, 32, 140, 300, 280, 150, 90, 85, 80]

temperature = [12, 14, 18, 24, 30, 34, 32, 31, 28, 22, 16, 13]

Label both y-axes and use different colors for bars and the line

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']
rainfall = [78, 62, 55, 41, 32, 140, 300, 280, 150, 90, 85, 80]
temperature = [12, 14, 18, 24, 30, 34, 32, 31, 28, 22, 16, 13]

fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.bar(months, rainfall, color='skyblue', label="Rainfall (mm)")
ax1.set_xlabel("Months")
ax1.set_ylabel("Rainfall (mm)", color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

ax2 = ax1.twinx()

ax2.plot(months, temperature, color='red', marker='o', linewidth=2,
label="Temperature (°C)")
ax2.set_ylabel("Temperature (°C)", color='red')
ax2.tick_params(axis='y', labelcolor='red')

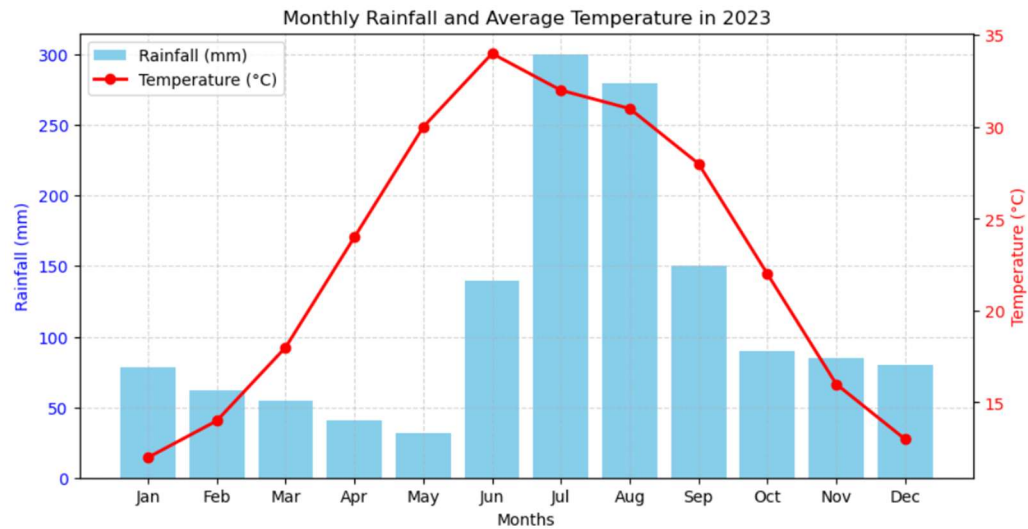
plt.title("Monthly Rainfall and Average Temperature in 2023")

ax1.grid(True, linestyle="--", alpha=0.5)

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper left")

plt.show()
```

output:



7: Draw a house using matplotlib line charts:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

# Walls (Rectangle)
walls_x = [1, 5, 5, 1, 1]
walls_y = [1, 1, 4, 4, 1]
plt.plot(walls_x, walls_y, color="brown", linewidth=2, label="Walls")

# Roof (Triangle)
roof_x = [1, 3, 5]
roof_y = [4, 6, 4]
plt.plot(roof_x, roof_y, color="maroon", linewidth=2, label="Roof")

# Door (Rectangle)
door_x = [2, 3, 3, 2, 2]
door_y = [1, 1, 3, 3, 1]
plt.plot(door_x, door_y, color="blue", linewidth=2, label="Door")

# Window 1 (Top Window)
win1_x = [4, 4.5, 4.5, 4, 4]
win1_y = [3, 3, 3.5, 3.5, 3]
plt.plot(win1_x, win1_y, color="teal", linewidth=2, label="Window 1")

# Window 2 (Bottom Window)
win2_x = [4, 4.5, 4.5, 4, 4]
win2_y = [1.5, 1.5, 2, 2, 1.5]
plt.plot(win2_x, win2_y, color="purple", linewidth=2, label="Window 2")

plt.title("House Using Line Plots with Multiple Colors")
```

```
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")  
plt.grid(True)  
plt.legend(loc="upper right")
```

```
plt.xlim(0.5, 5.5)  
plt.ylim(0.5, 6.5)
```

```
plt.show()
```

output:

